



OpenPinch: An Open-Source Python Library for Advanced Pinch Analysis and Total Site Integration

Timothy Gordon Walmsley¹

Received: 9 October 2025 / Revised: 16 February 2026 / Accepted: 2 March 2026
© The Author(s) 2026

Abstract

Pinch analysis provides a systematic approach to targeting the minimum heating and cooling requirements of industrial processes and to designing heat exchanger networks that approach this target. Decades after its introduction, the method remains essential for energy efficiency, decarbonisation, and retrofit studies across sectors such as chemicals, pulp and paper, food, and refining. In addition, numerous papers have proposed extensions to PA but these are often left isolate and unavailable for future development and implementation. In short, tool development has not kept pace with academic progress and today's industry requirements. Many practitioners rely on bespoke or basic spreadsheets or legacy desktop applications that make it difficult to scale analyses, embed the results in modern workflows, or experiment with emerging pinch techniques. OpenPinch fills this gap with an open-source Python toolkit that implements advanced pinch analysis and total site integration methods. The library offers typed schemas for robust data ingestion, modular analysis routines that span unit operation to process, site and regional scales, and export paths that maintain compatibility with conventional spreadsheet workflows. As illustrated by a survey of current pinch tools, a distinctive contribution is its comprehensive implementation of (unified) total site heat integration combined with multiple utilities, both isothermal and non-isothermal ones, via a combination of problem-table transformations, temperature-interval segmentation, and iterative assignment. The basis of these algorithms were reported in Tarighaleslami et al. (2017a). This technical note documents the architecture, algorithms, and data flow of OpenPinch, highlighting an end-to-end example of a pulp mill case study as well as key directions for future extension.

Keywords Pinch analysis · Total site integration · Process integration

Introduction

Energy systems integration across multiple scales presents a fundamental shift in how industrial, community and regional energy networks are conceived, designed and operated (Lee et al. 2020). Integration, in this context, is not the mere combining of systems and scales; instead, true integration attempts to uncover novel synergies that result from designing individual parts to achieve system level objectives. From an industrial perspective, Pinch Analysis (Linnhoff and Flower 1978) and Total Site Analysis (Klemeš et al.

1997) provide powerful methods for identifying energy savings and optimisation within process zones and industrial utility systems (Walmsley et al. 2023). These methods focus on thermal energy and are grounded in first-principles thermodynamics, delivering clear minimum utility targets and guiding heat exchanger network design.

While these site-focused approaches remain valuable, they require significant modification to address contemporary energy systems challenges. The emergence of variable renewable energy (VRE) sources, distributed generation, and sector coupling—linking the electricity, heating and cooling, and fuel sectors—necessitates new paradigms of integration that span spatial, temporal, and organisational boundaries. Locally integrated energy sectors (Perry et al. 2008) and industrial symbiosis (Chertow 2008) attempted to bridge this gap using an industry-centred approach. These recognise the value of heat and mass integration in industry and neighbouring energy users, shared infrastructure, and

✉ Timothy Gordon Walmsley
tim.walmsley@waikato.ac.nz

¹ School of Engineering, Ahuora – Centre for Smart Energy Systems, University of Waikato, Hamilton 3240, New Zealand

heat and power cogeneration, but comprehensive tools to underpin the contemporary analysis needed to address these emerging challenges remain a significant gap. In particular, there is no widely adopted analytical framework that supports energy system integration and design across the scales from reactors and separators through to regional energy systems using a consistent methodology and abstraction.

However, adopting, extending and applying cutting-edge pinch workflows face several barriers. The application of new methods are often manually done for one-off case studies, making new case studies, sensitivity analysis and optimisation cumbersome. The bespoke nature of academic studies means it is challenging to apply multiple concepts to a single case study. In this environment, a modern, programmable pinch analysis tool is valuable. It must ingest machine-generated data, support automated studies, and remain transparent for engineering review.

In 2010, OpenPinch began as an Microsoft Excel™ workbook with a comprehensive suite of VBA macros. It was created in response to the lack of comprehensive pinch analysis spreadsheet tools available at the time. Since then, the spreadsheet approach proved extremely practical in both academic and industrial contexts, encapsulating advanced techniques such as multi-scale targeting, assisted heat integration, and multiple utility handling. Over time, the workbook grew complex, and version management became challenging. Although it has been shared widely and freely, the code lacked clarity and documentation, creating a barrier for others to conduct further research and development. The Python implementation re-imagines the workflow with the following goals:

- Preserve proven algorithms while migrating to a maintainable, test-friendly code base.
- Adopt an well-designed and extensible library structure with documented classes and functions for further research, development and application.
- Expose a clear API for integration into broader analytics pipelines.
- Support multiple data sources, including JSON, CSV bundles, and Excel, with strong input validation.
- Maintain compatibility with Excel to make interaction smoothly for engineers with minimal programming experience.

This technical note introduces OpenPinch as a Python package available through PyPI (Walmsley 2025a) or open source through GitHub under an MIT license (Walmsley 2025b). Together, these fulfil the need for an extensible platform with comprehensive documentation and implemented in the popular scientific computing language, Python. From this platform, new researchers entering the process integration

community can launch, test, and extend their own research through repository forks or additions to the primary repository. They can tackle the emerging challenges in energy systems using a multiscale approach underpinned by the power pinch methodology.

Beyond describing what the library does, this note explains how the functions interact, the rationale behind specific algorithms, and the architectural choices enabling extension. A case study is presented to demonstrate how a user interacts with the package.

Background and Related Work

Classical Pinch Analysis

Pinch analysis was invented by Bodo Linnhoff and colleagues (Linnhoff and Flower 1978; Linnhoff and Hindmarsh 1983), who showed how to aggregate process streams into composite curves, identify pinch points, calculate minimum utility consumption, and design effective heat exchanger networks. The approach relies on organising stream data into temperature intervals, computing heat cascades, and obeying pinch rules such as “do not transfer heat across the pinch.”

Extensions and Advanced Topics

In addition to classic pinch analysis, early and critical extensions included:

- **Total site integration:** Aggregating multiple process units or plants into shared utility systems (Dhole and Linnhoff 1993; Klemeš et al. 1997; Linnhoff and Dhole, 1993), accounting for geographical constraints, stream variability (Liew et al. 2013, 2014), and non-isothermal utility (Tarighaleslami et al. 2017a).
- **Utility targeting and network design:** Handling multiple utilities with different temperature levels, costs, or availability profiles, including utility system optimisation (Tarighaleslami et al. 2017b) and network design (Tarighaleslami et al. 2018).
- **Power and cogeneration integration:** Coupling heat and power analyses to understand interactions with turbine networks (Sun et al. 2014, 2015), including under uncertainty (Sun et al. 2017).
- **Heat pump integration:** Introducing devices such as heat pumps or vapour recompression to shift heat across the pinch (Townsend and Linnhoff, 1983a, 1983b).
- **Exergy analysis:** Evaluating the quality of energy flows and potential losses beyond simple heat balances (Ham-sani et al. 2018; Walmsley 2023).

OpenPinch draws from many of these developments, particularly in total site, utility targeting and multi-scale analysis.

In addition, its worthy note that numerous non-heat pinch application concepts have also been proposed, including water (Foo 2009), carbon emissions (Tan et al. 2009), hydrogen (Bealing and Hutton 2002), and power pinch analyses (Mohammad Rozali et al. 2014). For general directions of pinch methodology, readers are encouraged to consider Klemeš et al. (2018).

Tool Landscape

A survey of available pinch tools for heat integration reveals several categories with a comprehensive comparison in Table 1:

1. Commercial packages, such as Aspen Energy Analyzer (“aspenONE® Engineering,” 2025), offering robust capabilities but limited customisation, extensibility and integration depth without licensing.
2. Online tools, such as Energy (2021), that are free-to-use but not open-source, contain only classic pinch analysis and slow to scale to large stream data problems.
3. Academic prototypes developed in MATLAB, GAMS, or custom languages, often focusing on specific extensions with limited documentation and extensible structure.
4. Spreadsheet solutions, like the OpenPinch workbook (2022), that provide transparency but lack a fully-featured, modern programming language and implementation.

5. Open-source tools such as PyPinch (Nicusan 2025), typically aimed at educational use with basic pinch analysis.

OpenPinch occupies a critical space: translating a mature, industry-tested Excel workbook into an open-source Python package while preserving advanced algorithms from pinch analysis with comprehensive utility targeting through to total site integration and exergy analysis. The combination of Pydantic validation (Colvin et al. 2025), modular architecture, and export compatibility positions it well for both practitioners and researchers.

OpenPinch: Package Description and Methods

This section provides detailed insight into the terminology, architecture, structure, classes, and functions that are contained in the OpenPinch repository.

Key Terms, Classes, and Objects

Pinch-based targeting in OpenPinch relies on a shared vocabulary that spans heat exchange theory, optimisation targets, and the software abstractions used to automate studies. Defining and clarifying terms, as presented in Table 2, prior to the methodological details is critical for cross-referencing jargon and understanding the meaning of potentially ambiguous words. It also affords the opportunity to link terms with class names and describe their role.

Table 1 Survey of current pinch software tools for heat integration

Medium	Aspen Energy Analyzer	Supertaget	TLK Energy	PinCH	PyPinch	OpenPinch - Excel	OpenPinch - Python
	Desktop application	Desktop application	Web application	Desktop application	Python script	Excel Workbook	Python library
Cost for non-commercial use	Licensed	Licensed	Free	Licensed	Free	Free	Free
Cost for commercial use	Licensed	Licensed		Licensed	Free	Free	Free
Code base	Proprietary	Proprietary	Proprietary	Proprietary	Open-source	Open-source	Open-source
Core features							
Process heat integration targets	✓	✓	✓	✓	✓	✓	✓
Problem table & heat cascade	✓	✓	✓	✓	✓	✓	✓
Composite curves and grand composite curve	✓	✓	✓	✓	✓	✓	✓
Isothermal utility placement	✓	✓	✗	✓	✓	✓	✓
Non-isothermal utility placement	✗	✗	✗	✓	✗	✓	✓
Total site problem table & heat cascade	✗	✗	✗	✗	✗	✓	✓
Total site heat integration targets	✗	✗	✗	✗	✗	✓	✓
Indirect source and sink targeting	✗	✗	✗	✓	✗	✗	✗

Table 2 Key terms used in OpenPinch and their meaning

Term	Meaning	Meaning in OpenPinch
Stream	A fluid that requires heating or cooling to be included in the analysis	A class containing the temperatures, pressures, flow rate and duty requirement of a stream
Stream data	A set of streams included in the pinch analysis	A class called StreamCollection, designed to house that contains a set of stream object
Hot stream	Process stream that must be cooled; potential heat source	Inferred from input temperatures to a Stream object with <code>is_process_stream=True</code> , assigns enum StreamType.Hot
Cold stream	Process stream that requires heating; potential heat sink	Inferred from input temperatures to a Stream object with <code>is_process_stream=True</code> , assigns enum StreamType.Cold
Hot utility	External heating medium supplying deficit above the pinch (e.g., steam)	Inferred from input temperatures to a Stream object with <code>is_process_stream=False</code> , assigns enum StreamType.Hot
Cold utility	External cooling resource removing excess heat below the pinch (e.g., cooling water)	Inferred from input temperatures to a Stream object with <code>is_process_stream=False</code> , assigns enum StreamType.Cold
Zone	Logical boundary (process, site, region) over which pinch targets are computed	Zone class underpins the hierarchical structure of targeting analysis across the scales.
Target	A theoretical or practical level of heat integration (heat recovery, utility use, etc.) that is achievable	EnergyTarget class captures all the heat integration targets relevant to a specific zone (system boundary)
Pinch point (temperature)	Temperature(s) where there is a thermodynamic bottleneck that prevents further heat integration	Anchor for identifying feasible heat matches and logging <code>hot_pinch</code> and <code>cold_pinch</code> temperatures.
Minimum approach temperature ($\Delta T_{\min}$)	Smallest temperature difference allowed between hot and cold streams in an exchanger	Inferred from $\Delta T_{\min, \text{cont}}$
Minimum approach temperature contribution ($\Delta T_{\min, \text{cont}}$)	Temperature shift of a hot or cold stream, contributing to the overall minimum approach temperature for heat transfer	User defined value (or default value) for the temperature shift when creating a stream
Composite curve	Plot of cumulative enthalpy vs. temperature for hot and cold process streams	Generated from problem table results to visualise recovery potential and $\Delta T_{\min}$ choices
Grand composite curve	Net heat-flow profile after pinch adjustment, highlighting utility requirements by temperature	Used to set utility-level targets and explore shifting loads between resource tiers
Total site profiles	Coordinated pinch analysis across multiple units sharing utilities	OpenPinch aggregates Zone results to evaluate site-wide (i.e., parent Zone) balancing, heat sharing and utility use
Site utility grand composite curve	Net heat-flow profile of utility streams	Aggregate utility use over the collection of zones
Problem table (or heat cascade)	Tabular method that marches temperatures and enthalpies to enforce $\Delta T_{\min}$ and locate pinch	Implemented as <code>problem_table_algorithm</code> , forming the backbone of most target calculations

Multi-Scale Characterisation

The package adopts a future-focused zone characterisation. Effective multiscale energy integration necessitates the decomposition of the system into nested levels and areas of analysis, each characterised by distinct thermodynamic constraints, technological options, and integration opportunities and objectives. This approach mirrors total site integration. In OpenPinch, five primary scales, or areas, are defined, from smallest to largest: unit, zone, site, community, and region. As a nested structure, each scale often encompasses one or more subsystems at smaller scales.

- Unit operation:** At the most granular level, integration focuses on the thermal and operational coupling within individual unit operations, such as reactors, distillation columns, evaporators, or dryers. Heat integration is typically achieved via internal design features and the optimisation of process conditions (e.g., set-points for temperature, pressure, and flow).
- Process:** The process (or utility zone) scale encompasses a collection of unit operations within a production area or plant section. The collection of units within a zone are often either production-focused or utility-generation-focused. Here, the attention shifts towards the synthesis of appropriate networks of unit operations, the design of heat exchanger networks (HENs), local utility generation and use, and integration of storage to buffer and manage demand fluctuations.
- Site:** Integration at the site level introduces the full complement of required utilities with their support systems, including combined heat and power cogeneration, centralised utility boilers and heat pumps, and site-wide storage infrastructure. Heat integration at the site level, or Total Site Heat Integration, occurs as heat recovery within the utility distribution network. Strategic storage deployment enables greater responsiveness and decouples production scheduling from instantaneous utility demand, a key consideration when interfacing with external variable renewable or price-varying grid-based energy sources.

- **Community:** At this scale, integration moves beyond single industrial entities to encompass clusters of facilities or mixed-use developments. These clusters form the community scale. Energy systems design accommodates diverse and decentralised users and producers, including residential, commercial, and industrial loads. Technologies such as district heating/cooling, waste-to-energy systems, and peer-to-peer energy trading are critical enablers.
- **Region:** The regional scale, including national and international trade, introduces systemic challenges relating to long-distance transmission, centralised generation, and large-scale energy balancing. Integration efforts must consider macro-scale supply chain logistics, siting of renewable and fossil-based plants, and the design of transmission and distribution networks. Energy carriers such as electricity, hydrogen, and synthetic fuels facilitate the transfer of energy across vast spatial domains and diverse demand centres. At this level, decisions are heavily influenced by policy, land use, resource availability, and economic geography.

Both community and regional level targeting analysis has yet to be implemented into OpenPinch. These are planned directions for future research and development.

Architecture

The architecture separates concerns deliberately as shown in Table 3:

- Domain objects (streams, zones) capture state and provide helper methods.
- Analysis modules maintain pure functions where possible, easing testing.
- Scales modules encompasses the base calls for targeting at the various scales.
- Schema and configuration provide a contract between inputs and algorithms.
- Utilities encapsulate I/O and cross-cutting concerns (timing, plotting).

This design aids extensibility, allowing contributors to insert new analyses, adapt export formats, or integrate additional data sources.

Data Model and Validation Layer

The enumeration types file in lib defines enumerations such as ‘StreamType’ (Hot, Cold, Both, Unassigned), ‘ZoneType’ (Process, Site, Community, Region, Utility), ‘TargetType’, and numerous ones relating to problem table columns. Using enumeration types reduce typos and ensures consistent keys across modules.

The configuration file also declares constants like ‘tol’ (a small, global tolerance) to avoid repeated magic numbers in numerical comparisons. This intolerance to implicit assumptions improves readability and maintainability. The ‘Configuration’ class holds default values for temperature corrections, zone naming, and feature toggles (e.g., whether to allow assisted integration). When ‘prepare_problem_structure’ builds the master zone, it instantiates ‘Configuration’ with user options, ensuring all downstream computations reference consistent settings. The configuration object is passed down to subzones, enabling features like site-wide assumption sharing.

In the schema file, key Pydantic models include:

- ‘ValueWithUnit’: Value and units for unit-aware calculations.
- ‘StreamSchema’: Defines zone affiliation, name, thermal properties, and status (‘active’ flag).
- ‘UtilitySchema’: Similar to streams but includes utility type, price, and optional heat flow (supporting cases where capacity is determined dynamically).
- ‘ZoneTreeSchema’: Recursive structure capturing hierarchical relationships.
- ‘Options’: Collections of enumerated options and turbine data.
- ‘TargetInput’: Entry point combining the above for validation.
- ‘TargetResults’, ‘TargetOutput’: Structures for outputs, including nested graph definitions.

Table 3 Package structure

Location	Purpose
OpenPinch/main.py	Orchestrates full analyses through ‘pinch_analysis_service’, wraps ‘get_targets’
OpenPinch/classes/	Contains domain objects (‘Stream’, ‘Zone’, ‘ProblemTable’, ‘PinchProblem’, etc.)
OpenPinch/scales/	Invokes a series of targets across the scales: unit operation, process, site, community, and region, using recursive loops.
OpenPinch/analysis/	Implements the heavy lifting: data preparation, process targeting, utility targeting, site/regional aggregation, response packaging, and supplementary analyses.
OpenPinch/utills/	Offers utilities for data conversion (Excel/CSV), decorators (timing), plotting helpers, and water property correlations.
OpenPinch/lib/	Houses enumerations, configuration, and Pydantic schema describing valid inputs and outputs

Pydantic ensures type coercion (e.g., reading strings and converting to floats where appropriate) and surfaces descriptive errors if data is missing or malformed. Early validation yields determinism by catching errors before algorithms run, producing reproducible failure modes; it improves safety by reducing the silent failures that arise when spreadsheets or external systems contain unnoticed mistakes; and it enhances integration readiness because third-party applications can rely on the schema to generate valid data and interact programmatically without manual intervention.

Creating and Running a Pinch Problem

The primary entry point to the package are for users to call instantiate a `PinchProblem`. Its initialisation accepts raw data housed in either a dictionary, JSON-like structure, or Pydantic model that define the zone hierarchy, streams, and utilities. `PinchProblem` is primarily designed as a light wrapper that encapsulates a call to `'pinch_analysis_service'`. The high-level workflow of this function proceeds as follows:

1. **Input validation:** `'TargetInput'` ensures streams, utilities, options, and optional zone tree meet structural requirements. Validation occurs before any computations, reducing runtime errors.
2. **Problem preparation:** The `'prepare_problem'` method encompasses the core methods to build the problem. It begins with validated zone information, merges defaults with user options through a `'Configuration'` object to determine the types of analysis that will run, builds the zone hierarchy, converts `StreamSchema` entries into `'Stream'` objects, and allocates utilities.
3. **Targeting:** Depending on the zone type (unit operation, process, site, community, region), specific analysis functions are invoked recursively. These routines calculate problem tables, identify pinch points, incorporate utility targeting, and are easily extendable to new concepts.
4. **Results extraction and response validation:** A light wrapper recursively grabs results (targets, utilities, and graphs) from each Zone within the problem for reporting to the user. `'TargetOutput'` can then validate the results follow the defined Pydantic output structure.

Once complete, the user can export the results to Excel using `'PinchProblem.export'`.

Process Stream and Problem Preparation

Prior to targeting, input data need validation, organisation and pre-processing. The entry point for this analysis is `'prepare_problem'`. This function starts by determining the root

zone name and type. If a zone tree is provided, it interprets entries `"Unit"`, `"Process"`, `"Site"`, `"Community"`, or `"Region"` and maps them to `'ZoneType'` enumeration type. Zone tree encapsulates the hierarchical structure of the problem. Without a tree (`ZoneTree`), it treats the project as a single zone of identifier `Site`, named after the project, with possible zones of identifier `Process`, which sit under the `Site`.

It then validates the inputs (streams, utilities) via a series of helper functions. Using the validated tree, the functions recursively builds `'Zone'` objects, storing them in dictionaries for quick lookup. Each `'Zone'` tracks subzones, parent zone, streams, utilities, and targets. The top-level zone is referred to as the master zone.

With the zone structure prepared, the next step is to assign streams to zones. The function `'_get_process_streams_in_each_subzone'` achieves this goal in the following way:

1. It generates zone paths (e.g., `'Site/Unit/Area'`) by traversing parent pointers.
2. For each stream, it compares the stream's zone field (which may be a path or a simple name) to available zones, using suffix matches to handle abbreviated forms.
3. Streams are instantiated via `'_create_process_stream'`, which sets attributes such as `'CP'`, `'heat_flow'`, and `'is_process_stream'`, then added to the zone's `'StreamCollection'`.

`'StreamCollection'`, a custom class, handles duplicates by appending numerical suffixes, maintains lazy sorting, and provides methods for retrieving indices or merging collections.

Utilities undergo similar processing. `'_get_hot_and_cold_utilities'` assigns the user defined utility streams to each zone object. A validation of these utility streams ensures any missing entries (e.g., outlet temperature of a steam utility) are inferred and appropriately. It also calculates extreme process temperatures (minimum hot stream supply, maximum cold stream target) to confirm the defined utilities have the ability cover the entire heating or cooling load. If not, it creates suitable default utility streams.

Utilities are then propagated to zones via `'_set_utilities_for_zone_and_subzones'`, ensuring each zone inherits the necessary context. Zones store both hot/cold utilities and net hot/cold streams to simplify later targeting steps.

After preparation, the master zone includes:

- Subzones representing the site hierarchy.
- Populated `'StreamCollection'` objects for hot and cold process streams.
- Utility collections ready for targeting.
- Configuration embedded for consistent downstream behaviour.

This stage is critical; any issues here ripple throughout the analysis. The modular functions and strong validation make debugging and extension manageable.

Targeting

Computing thermodynamic based targets is the goal of OpenPinch. It does this through two core methods: direct and indirect heat integration. Direct heat integration follows classical pinch analysis (Linnhoff 1994) while indirect heat integration follows the unified total site method proposed by Tarighaleslami et al. (2017a). This total site handles both isothermal (e.g., steam) and non-isothermal (e.g., water) utilities. There are also several supplementary targeting methods such as for area, unit, and exergy targeting at varying stages of implementation. However, this paper focuses on the two core methods of direct and indirect heat integration.

Direct Heat Integration

Direct integration refers to the recovery of heat by exchanging energy directly between hot and cold process streams, without the need for intermediate utilities (such as steam or cooling water). This is the central strategy for achieving maximum energy efficiency with processes because streams are generally spatially close and operate with similar timing. Classical pinch analysis through the problem table algorithm determines direct heat integration targets.

It is critical understand the inherent assumptions. Streams need defined temperatures and flows to determine their heating and cooling requirements. A minimum approach temperature contribution, ΔT_{cont} , must be inputted for each stream, or a global minimum approach temperature selected. Any hot stream must be able to transfer heat to any cold stream within the zone.

Streams with variable heat capacity between the supply and target temperatures require piece-wise linearisation (available via the ‘get_pieewise_linearisation_for_streams’ as a bespoke function). The goal of this function is to convert nonlinear stream T-H curves into a few breakpoint that can be used to created piecewise-linear segments.

Steps:

- (1) **Entry:** validates each stream has a matching dataset and fixes the start and end points of the piecewise-linear segments to be the supply and target temperatures of the hot or cold stream,
- (2) **Looping criteria:** creates a while loop with tolerance-driven criteria (maximum temperature deviation, dt_diff_max) to check the piecewise-linear segments against the non-linear curve,
- (3) **RDP application:** applies the well-known Ramer–Douglas–Peucker, RDP, method to drop redundant points,

- (4) **Refinement via optimisation:** refines breakpoint sets with a constrained optimisation that is solved using the SLSQP method from SciPy to enforce feasible hot/cold stream behaviour, i.e., a linearised hot stream sits below the original curve or a linearised cold stream sits above the original curve,
- (5) **Exit or loop:** iterates, if needed, until errors stay under the user’s dt_diff_max , returning the refined breakpoints for every stream, from which linear streams are easily deduced.

Direct heat integration is targeted when calling ‘get_process_heat_cascade’, the cornerstone for process-level analyses, followed by ‘get_utility_targets’, the method for multiple utility targeting (including non-isothermal utilities). The utility targeting approach is a comprehensive implementation of the algorithm of Tarighaleslami et al. (2017a). The heat cascade requires passing in collections of hot and cold streams, an additional collection with all process and utility streams, and the configuration object.

Steps:

1. **Extract temperature intervals:** The supply and target temperatures of all process and utility streams form the descending ordered temperature intervals. Two sets of intervals are extract; one based on shifted temperatures, and one based on the real process temperatures.
2. **Compute the problem table:** The problem table algorithm is applied twice based on real and shifted temperatures, together with the stream heat duties.
3. **Set targets:** From the problem table, it is easy to read-off and calculate heat recovery and utility targets for the zone. The pinch temperature(s) is also determined.

If the zone has subzones, the function processes them recursively. This software design ensures that each level of the hierarchy receives appropriate targeting results without redundant code.

Many extensions to the current implementation are possible. For example, dynamic pinch analysis (Walden et al. 2023) could be added by running the direct heat integration method multiple time, allowing multi-period analysis and time dependent stream changes.

Indirect Heat Integration

Indirect heat integration refers to the recovery and transfer of energy between processes using intermediate systems such as utility networks, rather than through direct heat exchange between process streams. This is the approach taken in total site integration and locally integrated energy sectors (i.e., the community level). As a result, indirect integration

encompasses entire industrial complexes or multi-site systems, where centralised energy hubs, combined heat and power systems, or large-scale thermal energy storage coordinate energy flows across multiple sectors. Fundamentally, surplus heat from one system is redistributed to others where there are heat deficits at lower temperatures.

The hierarchical perspective allows for more flexible and scalable energy recovery strategies, where localised exchanges optimise within a plant or sector, and total site management coordinates these subsystems to minimise overall fuel use, emissions, and utility costs. The method in OpenPinch follows the work of Tarighaleslami et al. (2017a).

In OpenPinch, this workflow starts with ‘get_site_targets’, which ensures each site and its subzones already contain detailed process targets (get_process_targets). Those process targets capture the hot and cold utility demands, pinch temperatures, and composite curve data required for aggregation. Heat recovery pockets on the grand composite curve are removed, indicating the heat is retained in the process. Remaining segments, i.e., net heat surplus and deficit segments, are stored and are available to the site level zone.

The method for determining the amount of indirect integration is given in five steps. The problem table algorithm with utility stream inputs, instead of process stream inputs, sits at the heart of the approach, making reuse of the same function.

Steps:

1. **Simple summation of subzone targets:** This step extracts the targets from each subzone and sums them together to get site-wide totals. This summation does not consider any potential indirect heat integration.
2. **Import stream segments:** Net heat surplus and deficit segments from all subzones are imported into the site. These are the process stream segments that correspond to utility use.
3. **Site process cascade:** This reuses the ‘get_process_heat_cascade’ method from process level analysis but with the stream segments as the primary inputs. The main output is a problem table object that contains the composite curves that are the process side of the total site profiles.
4. **Site utility cascade:** This invokes a different wrapper called ‘get_utility_heat_cascade’ that also utilises the problem table algorithm function to compute the cascade of utility streams. This is essential for calculating the correct net amount of utility and the site utility grand composite curve.
5. **Set targets:** With all the process and utility cascades computed, it is easy to read-off and calculate heat recovery and utility targets for the site. Sites often have a pinch temperature range driven by two utility levels. Where a pinch range exists, OpenPinch finds the hot and cold pinch (shifted) temperatures (upper and lower sides).

Together, these steps let OpenPinch translate detailed process data into site-wide indirect heat integration insights, quantifying residual utility requirements after feasible utility-to-utility recovery and providing the targets metrics needed for decision-making.

Result Packaging and Export

The final part of targeting is to report the results. The method ‘output_response’ walks through the zone hierarchy, extracts relevant data, and constructs an output dictionary containing the results. It converts internal ‘StreamCollection’ objects into serializable forms and structures graphs according to the schema. The final output conforms to the ‘TargetOutput’ Pydantic model, ensuring downstream consumers (such as the Excel export tool) can rely on typed fields. Excel exports maintain parity with the original workbook, which is useful for seamless comparison and further analysis. Many organisations rely on Excel for analysis and reporting; the Python pipeline can feed into generating reports automatically. Another method to view the results is through the OpenPinch Dashboard, enabled by the streamlit and plotly Python packages. The next section presents screenshots from the current dashboard implementation.

Automated Testing and Validation

A rigorous end-to-end (E2E) testing framework is implemented using *pytest* to validate the complete OpenPinch workflow against a comprehensive suite of ten benchmark problems with known and validated results. The E2E test (test_main.py) executes the full analysis pipeline on the example cases, comprising a variety site and plant scenarios. These include hot-only and cold-only systems as well as illustrative, refinery and pulp mill cases from the literature. For each case, the computed results are compared numerically against corresponding validated results. Agreement is enforced using tiny tolerances on key outputs, including minimum hot, cold, and recovery utility targets, as well as individual utility duties. This approach ensures that both problem table analysis and utility targeting are stress-tested repeatedly across a diverse range of industrially relevant datasets.

Beyond the E2E tests, targeted unit and regression tests provide fine-grained coverage of the core computational components. All the core functions, including problem table construction and cascade logic, temperature interval handling, net heat capacity and enthalpy calculations, temperature interval insertion, utility targeting, stream linearisation, etc., have unit test to ensure that when changes to the code base are made, no deviation in targeting results occur. Collectively, this layered testing strategy—combining detailed unit checks with an E2E

pipeline—provides strong confidence that stream preprocessing, problem table generation, utility targeting, and final target aggregation reproduce the established benchmarks across a wide range of scenarios accurately.

Running OpenPinch

OpenPinch exposes two methods to users for interaction with the package. First, a user can create a PinchProblem object as a script-based approach to calculating heat integration targets. Second, a user can pass in a problem and view the results as a Streamlit dashboard with graphs generated using Plotly. In this section, details around both pathways are presented. It concludes with the example of a Pulp Mill from Bood and Nilsson (2013).

Script Calling

The script in Fig. 1 executes the targeting approach of OpenPinch. This performs the following:

1. Converts the Excel file into a JSON.
2. Loads the JSON file.
3. Validates data via ‘TargetInput’.
4. Prepares the zone hierarchy and streams.
5. Runs targeting.
6. Exports results to Excel under ‘results/pulp_mill’.

If ‘results_dir’ is omitted, the analysis still runs, and results stay in memory for programmatic access through the PinchProblem object. A user can also supply a Configuration object to manipulate which analyses run. For one-off problems, the calculation speed is normally less than a few seconds. As a result, passing in a Configuration is not needed for conventional use cases.

Once the PinchProblem is solved, it results a Pydantic model that contains several key outputs, including:

- Minimum hot utility (‘Qh’) and cold utility (‘Qc’) targets for the site and subzones.
- Heat recovery targets (‘Qr’) indicating potential savings.

Python

```
from pathlib import Path
from OpenPinch import PinchProblem

problem = PinchProblem(
    problem_filepath=Path("examples/OpenPinchWkbs/pulp_mill.xlsb"),
    results_dir=Path("results"),
    run=True,
)
```

Fig. 1 Python script to call OpenPinch and analyse a well-defined problem

- Utility allocation tables showing how demand is distributed across multiple utilities.
- Net stream segments for potential site-wide balancing or total site analysis.
- Graphs such as composite curves, shifted composite curves, grand composite curve.

OpenPinch Dashboard Powered by Streamlit and Plotly packages

Streamlit provides an accessible, Python-centric pathway for deploying interactive dashboards around OpenPinch’s thermodynamic targeting workflow. Because Streamlit executes scripts in a reactive, state-aware manner, each adjustment to the pinch analysis inputs—zone selection, utility data, or composite-curve parameters—triggers an immediate refresh of OpenPinch’s underlying calculations. Live input of stream data is not yet possible.

The tight coupling between user interface and OpenPinch’s analytical engine preserves methodological transparency: practitioners can inspect and interpret intermediate results such as problem tables, composite curves, and load profiles without leaving the browser session or learning a bespoke UI framework. Moreover, Streamlit’s widget library (e.g., drop-down selectors, tabs, downloadable artefacts) allows OpenPinch to surface configuration options that might otherwise remain buried in configuration files, thereby lowering the barrier for process engineers to explore alternative heat integration scenarios and interpret the resulting trade-offs.

Adopting Streamlit also aligns with reproducibility and dissemination goals in academic settings. The entire dashboard is version-controlled alongside OpenPinch’s computational logic, ensuring that future published case studies capture both the analytical procedures and the interface through which stakeholders from academia and industry interpret them. Furthermore, Streamlit’s compatibility with headless deployment targets—ranging from local notebooks to institutional servers—facilitates collaborative experimentation.

Researchers can also provision shared dashboards that expose OpenPinch outputs to domain experts who may not possess Python expertise, enabling mixed-method evaluations that combine quantitative metrics (e.g., heat-recovery targets, utility balances) with qualitative feedback on design decisions. Consequently, the Streamlit-based delivery layer functions not merely as a visualisation add-on but as an integral component in OpenPinch’s research-to-practice translation workflow.

Example Application: Södra Cell Värö Kraft Pulp Mill

Pulp mills are energy-intensive sites with significant heat integration potential. The dataset ‘examples/stream_data/p_pulp_mill.json’ encapsulates hot and cold streams across

key processes, along with utilities such as steam at different pressures and cooling water options.

The original data source is Bood and Nilsson (2013), and it was one of three datasets used in Tarighaleslami et al. (2017a). To keep this note brief, the readers can view the OpenPinch workbook version in the Github repository (see examples/OpenPinchWkbs folder for multiple examples).

The dashboard is easily run by modifying the passed in file name of the problem and using the command: `streamlit run streamlit_app.py`.

Figure 2 presents a screenshot of the OpenPinch Dashboard, which is a locally hosted `streamlit` app. It shows the total site curves for the pulp mill case study. The targets align with those calculated by Tarighaleslami et al. (2017a) and the OpenPinch Excel Workbook. The left side of the dashboard includes the overall heat integration targets and the individual utility targets.

Results Comparison

The results in Table 4 demonstrate that Open Pinch reproduces published case study problems with very close agreement in minimum hot and cold utility targets across both single process and total site case studies. For small illustrative

examples as well as large industrial datasets deviations are negligible and within tight numerical tolerances. This confirms that the Python implementation preserves the thermodynamic integrity of the original algorithms while extending them to multiple utility and total site targeting. The consistency across diverse problem types provides validation that OpenPinch can be reliably applied to both academic studies and practical industrial heat integration projects.

Extensions for Future Research and Development

There are many possible directions to extend OpenPinch. The list here raises many ideas that have been planned but yet to be actioned.

- Community & region scales (indirect integration):**
 Extend the current unit–process–site stack to community and regional analyses, potentially reusing the unified total site formulation with utility segmentation to enable cross-site heat exchange, shared storage, and hub-and-spoke utility networks. Generalise total-site constructs to include power, hydrogen, and synthetic fuels as mediated carriers; provide coupling artefacts

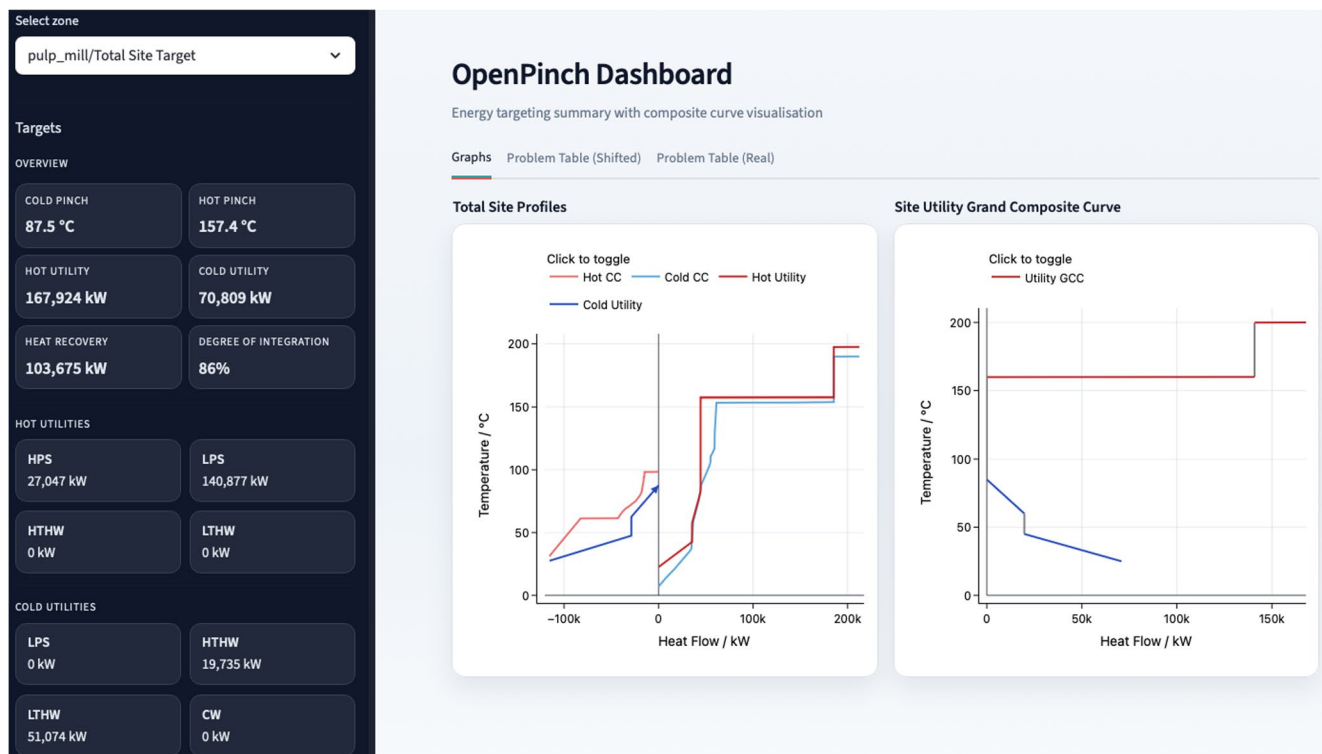


Fig. 2 OpenPinch dashboard – indicative output from the current streamlit app implementation and styling. The appearance of the streamlit app is expected to change as the OpenPinch community seeks to improve and expand the capabilities of the library

Table 4 Selected case studies from the literature showing good agreement with Open Pinch

Problem type	Case study details	Literature targets	OpenPinch results	Ref.
4-stream (2 H, 2 C)	Milk powder production process, involving an evaporator, pumps/fans/compressors, spray dryer, heater, cooler, and economiser.	$Q_h = 25.3 \text{ kW}$ $Q_c = 63.8 \text{ kW}$	$Q_h = 25.3 \text{ kW}$ $Q_c = 63.8 \text{ kW}$	(Kaviani et al. 2022)
4-stream (2 H, 2 C)	Simple example to demonstrate pinch analysis/HEN design with isothermal process streams.	$Q_h = 1000 \text{ kW}$ $Q_c = 1000 \text{ kW}$	$Q_h = 1000 \text{ kW}$ $Q_c = 1000 \text{ kW}$	(Ponce-Ortega et al. 2008)
7-stream (3 H, 4 C)	Example to demonstrate pinch analysis and heat exchanger network design where some streams have only latent heat, others have only sensible heat.	$Q_h = 5,106 \text{ kW}$ $Q_c = 1,847 \text{ kW}$	$Q_h = 5,106 \text{ kW}$ $Q_c = 1,847 \text{ kW}$	(Ponce-Ortega et al. 2008)
7-stream (4 H, 3 C)	Example like above, seven streams, all of which are isothermal.	$Q_h = 1,068.7 \text{ kW}$ $Q_c = 1,900 \text{ kW}$	$Q_h = 1,069 \text{ kW}$ $Q_c = 1,900 \text{ kW}$	(Ponce-Ortega et al. 2008)
6-stream (3 H, 3 C)	Example where two streams exchange only latent heat, two exchange only sensible, and two exchange both kinds of heat.	$Q_h = 1428.4 \text{ kW}$ $Q_c = 14588.3 \text{ kW}$	$Q_h = 1429 \text{ kW}$ $Q_c = 14,588 \text{ kW}$	(Ponce-Ortega et al. 2008)
8-stream (4 H, 4 C)	Streams with latent heat, sensible heat, or both.	$Q_h = 420.3 \text{ kW}$ $Q_c = 4982.4 \text{ kW}$	$Q_h = 420 \text{ kW}$ $Q_c = 4982 \text{ kW}$	(Ponce-Ortega et al. 2008)
23-stream (17 H, 6 C)	Pinch analysis used to identify integration improvement areas in a crude distillation unit. Elements of the CDU include pre-flash, fractionation, and heat exchange sections. Retrofit HEN example	$Q_h = 48.4 \text{ MW}$ $Q_c = 32.6 \text{ kW}$	$Q_h = 48.4 \text{ MW}$ $Q_c = 32.6 \text{ MW}$	(Mrayed et al. 2021)
Total site: 3 process zones	Heat integration project for a refinery, rubber plant, and tank field in China.	$Q_h = 1,720 \text{ kW}$ $Q_c = 16,780 \text{ kW}$	$Q_h = 1,722 \text{ kW}$ $Q_c = 16,778 \text{ kW}$	(Wang et al. 2014)
Total site 3 process zones	Aniline plant case study, with three of the four subsystems analysed for retrofit heat integration.	$Q_h = 9,148 \text{ kW}$ $Q_c = 16,368 \text{ kW}$	$Q_h = 9,148 \text{ kW}$ $Q_c = 16,378 \text{ kW}$	(Feng et al. 2011)
Total site 3 process zones (77 streams)	Aromatic hydrocarbon plant with hydrocracking unit, aromatic hydrocarbon extracting unit, and isomerization unit.	$Q_h = 63,875 \text{ kW}$ $Q_c = 122,097 \text{ kW}$	$Q_h = 63,875 \text{ kW}$ $Q_c = 122,097 \text{ kW}$	(Feng et al. 2011)

(P2H, H2 heat, CHP back-pressure envelopes) for locally integrated energy sectors.

- **Multi-period and dynamic targeting:** Add time-slicing and profile handling (loads, prices, energy availability, batch schedules) so direct and indirect targets can be computed across operating periods; expose schedule-aware composite and grand-composite curves. Incorporate thermal (sensible/latent) and limited electro-thermal storage at each analysis scale, with charge–discharge constraints and possibly round-trip losses embedded in targeting.
- **Heat-pump and cogeneration integration:** Provide native abstractions for electrically driven heat pumps, mechanical vapour recompression, and site cogeneration; support parametric studies and $\Delta T_{\min}$ -consistent placement rules across pinch. Exergy analysis could play a critical role in the targeting process.

Researchers are welcome to contact the corresponding author for further discussion on the various directions.

Conclusion

OpenPinch encompasses advanced pinch analysis and total site integration workflows as a modern Python library. By combining strict input validation, modular

analysis routines, and compatibility with legacy outputs, it bridges the gap between ad hoc engineering tools and programmable analytics platforms. The improved multi-utility targeting algorithm, capable of handling non-isothermal utilities through iterative segmentation, distinguishes OpenPinch from many alternatives. The pulp mill case study demonstrates practical application, showing how complex utility interactions can be quantified and exploited. Moving forward, community contributions for continued development are most welcome. Through community participation, OpenPinch aims to become a shared foundation for advanced pinch analysis research and practice.

Appendix A: Problem Table Algorithm

This function constructs the problem table (*pt*), which enters with defined temperature intervals, and associated heat cascades from a set of hot and/or cold process streams (*hot_streams*, *cold_streams*). It aggregates stream heat capacity data over temperature intervals, computes hot, cold, and net enthalpy cascades, and appropriately shifts these cascades to enforce the pinch condition. Its purpose is to establish a thermodynamically consistent foundation for pinch identification, composite curve construction and subsequent utility targeting.

FUNCTION PROBLEM_TABLE_ALGORITHM(

```

    pt,
    hot_streams = None,
    cold_streams = None,
    is_shifted = TRUE
)

/* Aggregate stream heat capacity data */
IF hot_streams ARE PROVIDED THEN
    SUM  $m \cdot C_p$  OVER temperature intervals
    USING shifted OR real temperatures
    STORE RESULTS IN:
        pt.CP_HOT
END IF

IF cold_streams ARE PROVIDED THEN
    SUM  $m \cdot C_p$  OVER temperature intervals
    USING shifted OR real temperatures
    STORE RESULTS IN:
        pt.CP_COLD
END IF

/* Compute temperature interval widths */
pt.DELTA_T  $\leftarrow$  DELTA_WITH_ZERO_AT_START(pt.T)

/* Hot composite heat cascade */
pt.DELTA_H_HOT  $\leftarrow$  pt.DELTA_T  $\times$  pt.CP_HOT
pt.H_HOT  $\leftarrow$  CUMULATIVE_SUM(pt.DELTA_H_HOT)

/* Cold composite heat cascade */
pt.DELTA_H_COLD  $\leftarrow$  pt.DELTA_T  $\times$  pt.CP_COLD
pt.H_COLD  $\leftarrow$  CUMULATIVE_SUM(pt.DELTA_H_COLD)

/* Net heat cascade */
pt.CP_NET  $\leftarrow$  pt.CP_COLD - pt.CP_HOT
pt.DELTA_H_NET  $\leftarrow$  pt.DELTA_T  $\times$  pt.CP_NET
pt.H_NET  $\leftarrow$  -CUMULATIVE_SUM(pt.DELTA_H_NET)

/* Pinch alignment: enforce minimum net enthalpy at zero */
min_H  $\leftarrow$  MIN(pt.H_NET)
delta_shift  $\leftarrow$  LAST(pt.H_NET) - min_H

/* Position composite curves */
pt.H_HOT  $\leftarrow$  LAST(pt.H_HOT) - pt.H_HOT
pt.H_COLD  $\leftarrow$  LAST(pt.H_COLD) + delta_shift - pt.H_COLD
pt.H_NET  $\leftarrow$  pt.H_NET - min_H

RETURN pt
END FUNCTION

```

Appendix B: Multiple Isothermal and Non-Isothermal Utility Targeting Algorithm

The utility targeting algorithm is split across three core functions, as described below with the corresponding pseudo-code.

- **TARGET_UTILITY**: This function serves as the entry point for performing utility targeting against a specified temperature-heat flow profile. It prepares the heat cascade

for targeting, determines whether utilities should be applied based on residual heat demand or surplus, and delegates the detailed allocation of utility duties to the assignment routine.

- **ASSIGN_UTILITY**: This function allocates utility duties across the temperature intervals on the appropriate side of the pinch. It selects the valid heat flow segment, enforces pinch constraints, iterates through utilities in priority order, and accumulates assigned heat until the required heating/cooling is assigned.

FUNCTION TARGET_UTILITY(

```

    utilities,
    T_vals,
    H_vals,
    hot_pinch_row,
    cold_pinch_row,
    is_real_temperatures = FALSE
)

IF utilities ARE HOT UTILITIES THEN
    IF H_vals AT START  $\neq$  0 THEN
        CALL ASSIGN_UTILITY(
            T_vals,
            H_vals,
            utilities,
            hot_pinch_row,
            is_hot_ut = TRUE,
            is_real_temperatures
        )
    END IF
ELSE /* cold utilities */
    IF H_vals AT END  $\neq$  0 THEN
        CALL ASSIGN_UTILITY(
            T_vals,
            H_vals,
            utilities,
            cold_pinch_row,
            is_hot_ut = FALSE,
            is_real_temperatures
        )
    END IF
END IF

RETURN utilities
END FUNCTION
```

```

FUNCTION ASSIGN_UTILITY(
    T_vals,
    H_vals,
    utilities,
    pinch_row,
    is_hot_ut,
    is_real_temperatures
)

IF is_hot_ut THEN
    SELECT SEGMENT:
        rows FROM start TO pinch_row
    SET segment_heat_limit  $\leftarrow$  FIRST(H_vals)
    ITERATE utilities IN REVERSE PRIORITY
ELSE
    SELECT SEGMENT:
        rows FROM pinch_row-1 TO end
    SET segment_heat_limit  $\leftarrow$  LAST(H_vals)
    ITERATE utilities IN FORWARD PRIORITY
END IF

Q_assigned  $\leftarrow$  0

FOR EACH utility IN utilities DO
    DETERMINE Ts, Tt FROM utility
    (shifted OR real temperatures)

    Q_max  $\leftarrow$  MAXIMISE_UTILITY_DUTY(
        T_segment,
        H_segment,
        Ts,
        Tt,
        is_hot_ut,
        Q_assigned
    )

    IF Q_max > tolerance THEN
        ASSIGN Q_max TO utility
        Q_assigned  $\leftarrow$  Q_assigned + Q_max
    END IF

    IF Q_assigned  $\geq$  segment_heat_limit THEN
        BREAK
    END IF
END FOR

RETURN utilities
END FUNCTION

```


FUNCTION MAXIMISE_UTILITY_DUTY(

T_segment,
H_segment,
Ts,
Tt,
is_hot_ut,
Q_assigned
)

IF LENGTH(T_segment) < 2 THEN

 RETURN 0

END IF

IF is_hot_ut THEN

 BUILD INTERVALS LOOKING DOWNWARD

ELSE

 BUILD INTERVALS LOOKING UPWARD

END IF

FOR EACH interval DO

$\Delta H_{\text{remaining}} \leftarrow (\text{cascade } \Delta H) - Q_{\text{assigned}}$

$\Delta T_{\text{supply}} \leftarrow T_s - T_{\text{interval}}$

$\Delta T_{\text{target}} \leftarrow T_{\text{interval}} - T_t$

END FOR

CREATE validity_mask WHERE:

$\Delta H_{\text{interval}} \neq 0$

$\Delta T_{\text{supply}} \geq 0$

$\Delta H_{\text{remaining}} > 0$

IF NO VALID INTERVALS THEN

 RETURN 0

END IF

$Q_{\text{ts_max}} \leftarrow \text{MAX}(\Delta H_{\text{remaining}} \text{ OVER valid intervals})$

$Q_{\text{tt_max}} \leftarrow \text{MIN}(\text{heat limited by reaching target temperature})$

IF $Q_{\text{tt_max}} \leq 0$ THEN

 RETURN 0

END IF

 RETURN MIN($Q_{\text{ts_max}}$, $Q_{\text{tt_max}}$)

END FUNCTION

• **MAXIMISE_UTILITY_DUTY:** This function determines the maximum feasible heat duty that a single utility can supply or absorb within a given temperature and heat flow range. It evaluates interval-wise heat availability, temperature approach constraints, and remaining cascade capacity to compute a limiting duty. Its purpose is to provide a physically constrained upper bound on utility contribution before assignment.

Wrapped around these functions is a top-level orchestration function for utility targeting, calling **TARGET_UTILITY** for placing either hot or cold utilities in any given call. The wrapper coordinates the data from various integration levels, such as process or site problem table data. It supplies the required temperature-heat flow load profile for heating or cooling (T_vals , H_vals), the specific hot or cold utilities available (*utilities*), pinch points (*hot_pinch_row*, *cold_pinch_row*, i.e., the location of the hot and cold side pinch temperatures in T_vals), and a flag (*is_real_temperatures*) to signal whether the temperature data has been shifted or not.

Acknowledgements The author thanks the following former and present members of the Ahuora Centre, including Alex Geary, Stephen Burroughs, Harrison Whiting, Ethan MacLeod, Benjamin Lincoln, Elsa Burroughs, Aleexa Adeele, and Keegan Hall for support in developing earlier versions of OpenPinch Python library from the Excel VBA codebase and/or contributing to discussions about OpenPinch and its application.

Author Contributions T.G.W.: conceptualisation, methodology, software, formal analysis, case study, writing - original draft, visualisation.

Funding Open Access funding enabled and organized by CAUL and its Member Institutions. This research has been supported by the programme “Ahuora: Delivering sustainable industry through smart process heat decarbonisation”, an Advanced Energy Technology Platform that is a collaboration between the University of Waikato, University of Auckland, and Massey University, funded by the New Zealand Ministry of Business, Innovation and Employment.

Data Availability The data that support the findings of this study are available from the Github repository OpenPinch: <https://github.com/waikato-ahuora-smart-energy-systems/OpenPinch>.

Declarations

Conflict of interest The author declares no conflict of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript, or in the decision to publish the results.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- aspenONE® Engineering [WWW Document] (2025). Aspen Technology. URL <https://www.aspentech.com/> (accessed 11.3.18)
- Bealing C, Hutton D (2002) Feature report: hydrogen-pinch analysis. *Chem Eng* 109:56–61
- Bood J, Nilsson L (2013) Energy Analysis of Hemicellulose Extraction at a Softwood Kraft Pulp Mill, Case Study of Södra Cell Värö (MSc Thesis). Chalmers University of Technology, Gothenburg, Sweden
- Chertow MR (2008) Uncovering industrial symbiosis. *J Ind Ecol* 11:11–30. <https://doi.org/10.1162/jiec.2007.1110>
- Colvin S, Jolibois E, Ramezani H, Garcia Badaracco A, Dorsey T, Montague D, Matveenko S, Trylesinski M, Runkle S, Hewitt D, Hall A, Plot V (2025) Pydantic Validation.
- Dhole VR, Linnhoff B (1993) Total site targets for fuel, co-generation, emissions, and cooling. *Comput Chem Eng* 17:S101–S109. [https://doi.org/10.1016/0098-1354\(93\)80214-8](https://doi.org/10.1016/0098-1354(93)80214-8)
- Energy TLK (2021) TLK Energy [WWW Document]. Pinch Analysis Tool. URL <https://tlk-energy.de/en/tools/pinch-analysis-online> (accessed 10.2.25)
- Feng X, Pu J, Yang J, Chu KH (2011) Energy recovery in petrochemical complexes through heat integration retrofit analysis. *Appl Energy* 88:1965–1982. <https://doi.org/10.1016/j.apenergy.2010.12.027>
- Foo DCY (2009) State-of-the-art review of pinch analysis techniques for water network synthesis. *Ind Eng Chem Res* 48:5125–5159. <https://doi.org/10.1021/ie801264c>
- Hamsani MN, Walmsley TG, Liew PY, Wan Alwi SR (2018) Combined pinch and exergy numerical analysis for low temperature heat exchanger network. *Energy* 153:100–112. <https://doi.org/10.1016/j.energy.2018.04.023>
- Kaviani A, Aslani A, Zahedi R, Ahmadi H, Malekli MR (2022) A new approach for energy optimization in dairy industry. No 8:100498. <https://doi.org/10.1016/j.clet.2022.100498>
- Klemeš JJ, Dhole VR, Raissi K, Perry SJ, Puigjaner L (1997) Targeting and design methodology for reduction of fuel, power and CO₂ on total sites. *Appl Therm Eng* 17:993–1003. [https://doi.org/10.1016/S1359-4311\(96\)00087-7](https://doi.org/10.1016/S1359-4311(96)00087-7)
- Klemeš JJ, Varbanov PS, Walmsley TG, Jia X (2018) New directions in the implementation of pinch methodology (PM). *Renew Sustain Energy Rev* 98:439–468. <https://doi.org/10.1016/j.rser.2018.09.030>
- Lee PY, Liew PY, Walmsley TG, Wan Alwi SR, Klemeš JJ (2020) Total site heat and power integration for locally integrated energy sectors. *Energy*. <https://doi.org/10.1016/j.energy.2020.117959>
- Liew PY, Wan Alwi SR, Klemeš JJ, Varbanov PS, Manan ZA (2013) Total site heat integration with seasonal energy availability. *Chem Eng Trans* 35:19–24. <https://doi.org/10.3303/CET1335003>
- Liew PY, Wan Alwi SR, Klemeš JJ, Varbanov PS, Abdul Manan Z (2014) Algorithmic targeting for Total Site Heat Integration with variable energy supply/demand. *Appl Therm Eng* 70:1073–1083. <https://doi.org/10.1016/j.applthermaleng.2014.03.014>
- Linnhoff B (1994) A User Guide on Process Integration for the Efficient Use of Energy. Institution of Chemical Engineers, Rugby, UK
- Linnhoff B, Dhole VR (1993) Targeting for CO₂ emissions for Total Sites. *Chem Eng Technol* 16:252–259. <https://doi.org/10.1002/cet.270160407>
- Linnhoff B, Flower JR (1978) Synthesis of heat exchanger networks: I. Systematic generation of energy optimal networks. *AIChE J* 24:633–642. <https://doi.org/10.1002/aic.690240411>
- Linnhoff B, Hindmarsh E (1983) The pinch design method for heat exchanger networks. *Chem Eng Sci* 38:745–763
- Mohammad Rozali NE, Wan Alwi SR, Abdul Manan Z, Klemeš JJ, Hassan MY (2014) Optimal sizing of hybrid power systems using

- power pinch analysis. *J Clean Prod* 71:158–167. <https://doi.org/10.1016/j.jclepro.2013.12.028>
- Mrayed S, Shams MB, Al-Khayyat M, Alnoaimi N (2021) Application of pinch analysis to improve the heat integration efficiency in a crude distillation unit. *Clean Eng Technol* 4:100168. <https://doi.org/10.1016/j.clet.2021.100168>
- Nicusan AL (2025) anicusan/PyPinch.
- Open Pinch, Consulting (2022) Toolkit Download [WWW Document]. Open Pinch Toolkit Download. URL <https://www.openpinch.com/toolkit-download> (accessed 10.3.25).
- Perry S, Klemeš JJ, Bulatov I (2008) Integrating waste and renewable energy to reduce the carbon footprint of locally integrated energy sectors. *Energy* 33:1489–1497. <https://doi.org/10.1016/j.energy.2008.03.008>
- Ponce-Ortega JM, Jiménez-Gutiérrez A, Grossmann IE (2008) Optimal synthesis of heat exchanger networks involving isothermal process streams. *Comput Chem Eng* 32:1918–1942. <https://doi.org/10.1016/j.compchemeng.2007.10.007>
- Sun L, Doyle S, Smith R (2014) Graphical cogeneration analysis for site utility systems. *Clean Technol Environ Policy* 16:1235–1243. <https://doi.org/10.1007/s10098-014-0742-7>
- Sun L, Doyle S, Smith R (2015) Heat recovery and power targeting in utility systems. *Energy* 84:196–206. <https://doi.org/10.1016/j.ene.2015.02.087>
- Sun L, Gai L, Smith R (2017) Site utility system optimization with operation adjustment under uncertainty. *Appl Energy* 186:450–456. <https://doi.org/10.1016/j.apenergy.2016.05.036>
- Tan RR, Sum Ng DK, Yee Foo DC (2009) Pinch analysis approach to carbon-constrained planning for sustainable power generation. *J Clean Prod* 17:940–944. <https://doi.org/10.1016/j.jclepro.2009.02.007>
- Tarighaleslami AH, Walmsley TG, Atkins MJ, Walmsley MRW, Liew PY, Neale JR (2017a) A unified total site heat integration targeting method for isothermal and non-isothermal utilities. *Energy* 119:10–25. <https://doi.org/10.1016/j.energy.2016.12.071>
- Tarighaleslami AH, Walmsley TG, Atkins MJ, Walmsley MRW, Neale JR (2017b) Total site heat integration: utility selection and optimisation using cost and exergy derivative analysis. *Energy* 141:949–963. <https://doi.org/10.1016/j.energy.2017b.09.148>
- Tarighaleslami AH, Walmsley TG, Atkins MJ, Walmsley MRW, Neale JR (2018) Utility exchanger network synthesis for total site heat integration. *Energy* 153:1000–1015. <https://doi.org/10.1016/j.energy.2018.04.111>
- Townsend DW, Linnhoff B (1983a) Heat and power networks in process design. Part I: criteria for placement of heat engines and heat pumps in process networks. *AIChE J* 29:742–748. <https://doi.org/10.1002/aic.690290508>
- Townsend DW, Linnhoff B (1983b) Heat and power networks in process design. Part II: design procedure for equipment selection and process matching. *AIChE J* 29:748–771. <https://doi.org/10.1002/aic.690290509>
- Walden JVM, Wellig B, Stathopoulos P (2023) Heat pump integration in non-continuous industrial processes by dynamic pinch analysis targeting. *Appl Energy* 352:121933. <https://doi.org/10.1016/j.apenergy.2023.121933>
- Walmsley TG (2023) Energy and exergy transfer diagrams for visualising flows in process systems. *Process Integr Optim Sustain.* <https://doi.org/10.1007/s41660-023-00384-1>
- Walmsley TG (2025a) OpenPinch: An advanced pinch analysis and total site integration toolkit [WWW Document]. URL <https://github.com/waikato-ahuora-smart-energy-systems/OpenPinch> (accessed 10.3.25)
- Walmsley TG (2025b) OpenPinch (Github) [WWW Document]. URL <https://github.com/waikato-ahuora-smart-energy-systems/OpenPinch> (accessed 10.3.25)
- Walmsley TG, Philipp M, Picón-Núñez M, Meschede H, Taylor MT, Schlosser F, Atkins MJ (2023) Hybrid renewable energy utility systems for industrial sites: a review. *Renew Sustain Energy Rev.* <https://doi.org/10.1016/j.rser.2023.113802>
- Wang Y, Feng X, Chu KH (2014) Trade-off between energy and distance related costs for different connection patterns in heat integration across plants. *Appl Therm Eng* 70:857–866. <https://doi.org/10.1016/j.applthermaleng.2014.06.012>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.